

Lava Release Tagging Guide

Operator manual for scripts/tag.sh. Read this before cutting your first release.

Table of Contents

1. [Overview](#)
2. [Tag format](#)
3. [What the script does](#)
4. [Source of truth: where versions live](#)
5. [Quick reference](#)
6. [Flag-by-flag reference](#)
7. [Common workflows](#)
8. [Adding a new app/service](#)
9. [Troubleshooting](#)
10. [FAQ](#)

Overview

Lava ships three independently versioned artifacts:

App key	Tag prefix	Source of truth	Description
android	Lava-Android	app/ build.gradle.kts (versionName / versionCode)	Android client (digital.vasic.lava.client)
api	Lava-API	proxy/ build.gradle.kts (apiVersionName / apiVersionCode)	Ktor proxy server (digital.vasic.lava.api)
api-go	Lava-API-Go	lava-api-go/ internal/version/ version.go (Name / Code)	Go API service (SP-2 successor to the Ktor proxy)

scripts/tag.sh automates the release process for either or both of them:

1. Reads the **current** versionName/versionCode from each app's source of truth.
2. Creates an annotated git tag in the canonical Lava format.
3. Pushes the tag to **every configured upstream** (GitHub, GitFlic, GitLab, GitVerse).
4. Bumps the source-of-truth files (versionName via semver rule; versionCode by +1).
5. Commits the bump and pushes the commit to every upstream.

The script is **idempotent** for tag creation: running it twice on the same release commit will skip already-existing tags (and warn) instead of failing.

Tag format

Lava-<App>-<versionName>-<versionCode>

Concrete examples:

Tag	Meaning
Lava-Android-1.0.0-1008	Android client, semver 1.0.0, code 1008
Lava-Android-1.0.1-1009	Android client, the next patch
Lava-API-1.0.0-1000	Proxy server, first tagged release
Lava-API-1.1.0-1001	Proxy server, minor bump
Lava-API-Go-2.0.0-2000	Go API service, first tagged release
Lava-API-Go-2.0.1-2001	Go API service, the next patch

Rules:

- <App> is one of Android, API, API-Go (case sensitive). Add new ones via the registry — see [Adding a new app/service](#).
- <versionName> is a strict three-component semver MAJOR.MINOR.PATCH. Pre-release suffixes (e.g. -rc1) are **not** supported by the bump logic; if you need one, run with --no-bump.
- <versionCode> is a positive integer. The script always advances it by +1 after a release.

What the script does

Per app (in registry order):

1. **Read current versions** from the build file (failing loudly if it can't parse them).

2. **Compose the tag** `Lava-<App>-<vname>-<vcode>`.
3. **Create an annotated tag** at HEAD with the message `Release <App> <vname> (versionCode <vcode>)`.
 - If the tag already exists locally, the step is **skipped** (warning printed) and the script continues.
4. **Push the tag** to each configured remote (one `git push <remote> refs/tags/<tag>` per remote).
5. **Compute new versions** (`--bump major|minor|patch` controls semver; `versionCode` is always +1).
6. **Write the bump** back into the build file and verify the change took effect (re-reads the value).

After all apps have been processed:

1. **Stage and commit** the bump with message `Bump versions after release: <apps> (--bump <part>)`.
2. **Push the commit** to each configured remote (`git push <remote> HEAD:<current-branch>`).

Pre-flight safety checks

Before any of the above runs:

- The working tree must be **clean** (`git status --porcelain` empty). Otherwise the script aborts.
- Every requested remote must exist (`git remote get-url`) — otherwise it aborts before any push.
- `--bump` value must be `major`, `minor`, or `patch`.
- `--app` value must be `android`, `api`, `api-go`, or `all`.

Dry run

`--dry-run` (or `-n`) prints every action the script *would* take, including the precise `git tag`, `git push`, and `sed` invocations, but performs no `git` mutations or file writes. Useful for rehearsals and code review.

Source of truth: where versions live

App key	File	versionName line	versionCode line
android	app/build.gradle.kts	versionName = "X.Y.Z" (inside defaultConfig)	versionCode = NNNN (inside defaultConfig)
api	proxy/build.gradle.kts	val apiVersionName =	val apiVersionCode = NNNN (top level)

App key	File	versionName line	versionCode line
		"X.Y.Z" (top level)	
api-go	lava-api-go/ internal/version/ version.go	Name = "X.Y.Z" (inside the const (...) block)	Code = NNNN (inside the const (...) block)

Both sets of identifiers are owned by tag.sh. **Do not edit them by hand for routine releases** — let the script bump them. Manual edits are appropriate only for cherry-picking a release version (e.g. cutting 1.1.0 deliberately) before running the script with --no-bump.

Quick reference

Print the help text.

scripts/tag.sh --help

Preview a full tag pass without touching anything.

scripts/tag.sh --dry-run

Cut a release for both apps, push everywhere, bump patch.

scripts/tag.sh

Cut Android only, bump minor.

scripts/tag.sh --app android --bump minor

Cut both apps locally, do not push.

scripts/tag.sh --no-push

Cut both apps and push only to GitHub.

scripts/tag.sh --remote github

Flag-by-flag reference

Flag	Argument	Default	Purpose
-h, --help	—	—	Print help and exit.
-n, --dry-run	—	off	Show actions; do not change anything.
-a, --app	android api api-go all	all	Restrict the run to a specific app.
--bump	major minor patch	patch	Which semver part to advance after tagging.

Flag	Argument	Default	Purpose
			versionCode is always +1.
--no-bump	—	bump on	Skip the post-tag bump and bump-commit entirely.
--no-push	—	push on	Tag/commit locally only; do not push.
--remote	remote name	github + gitflic + gitlab + gitverse (those that exist)	Push to a specific remote. Repeatable.

Unknown options cause the script to exit 1 with a hint to run --help.

Common workflows

1. Routine patch release of both apps

```
git switch master
git pull --ff-only
scripts/tag.sh --dry-run      # rehearse
scripts/tag.sh                # commit
```

After completion the repo will have:

- Two new tags pushed everywhere (e.g. Lava-Android-1.0.0-1008, Lava-API-1.0.0-1000).
- One new commit on master bumping both apps to the next patch.

2. Hot-fix the Android app only

```
scripts/tag.sh --app android --bump patch
```

The :proxy versions are untouched.

3. Cut a minor release for the API only

```
scripts/tag.sh --app api --bump minor
```

Result: Lava-API-1.0.0-1000 is created (current state), then apiVersionName becomes 1.1.0 and apiVersionCode becomes 1001.

4. Re-tag the same commit after you noticed a bad metadata change

If the previous run already pushed the tag, just bump manually or use `--no-push --no-bump` to rebuild the local tag, delete the stale one, and re-push. (The script does **not** force-push or move existing tags — that has to be a deliberate operator action.)

5. Tag locally, push later

```
scripts/tag.sh --no-push
# inspect tags / commits, then:
git push github --follow-tags
git push gitflic --follow-tags
git push gitlab --follow-tags
git push gitverse --follow-tags
```

6. Promote a manually edited versionName

If you want this release to be 2.0.0 exactly, edit `versionName` (and/or `apiVersionName`) by hand, commit, then run:

```
scripts/tag.sh --no-bump
```

The script will create `Lava-Android-2.0.0-<code>` (and `Lava-API-2.0.0-<code>`) at HEAD without bumping back down or up.

Adding a new app/service

When a new artifact joins the build, do all four steps:

1. **Add a source of truth** — pick a stable place for `versionName` and `versionCode` (e.g. `tools/build.gradle.kts` containing `val toolsVersionName = "X.Y.Z"`).
2. **Add reader/writer functions** in `scripts/tag.sh` modeled after `read_api_version_name / write_api_versions`.
3. **Add the app key** to `SUPPORTED_APPS` and extend the case `"$app"` in block to wire the reader, writer, and tag prefix.
4. **Update this guide** — add a row to the *Source of truth* table and to the *Tag format* examples.

Keep tag prefixes unique and human-readable (Lava-Tools, Lava-Web, Lava-CLI, ...). Avoid prefixes that collide with existing semver-only tags.

Troubleshooting

"Working tree is dirty"

The script refuses to run with uncommitted changes — that would entangle the bump commit with unrelated work. Fix with:

```
git status
git stash          # or git commit -m "wip"
scripts/tag.sh
```

"Failed to read ... versionName/versionCode"

The regex couldn't find the expected line. Most common causes:

- Someone reformatted app/build.gradle.kts and the versionName = "..." line no longer matches versionName *= *"...".
- The apiVersionName/apiVersionCode declarations were removed from proxy/build.gradle.kts.

Fix the source file or update the corresponding read_* regex in scripts/tag.sh. Always pair a regex change with a verifying dry-run.

"Tag ... already exists locally — skipping creation"

The script already created the tag in a previous run. If the previous run failed before the push completed, just re-run with the same arguments — the existing tag will be re-pushed.

To **delete** a stale tag (e.g. one that points to the wrong commit):

```
git tag -d Lava-Android-1.0.0-1008
git push github :refs/tags/Lava-Android-1.0.0-1008
git push gitflic :refs/tags/Lava-Android-1.0.0-1008
git push gitlab :refs/tags/Lava-Android-1.0.0-1008
git push gitverse :refs/tags/Lava-Android-1.0.0-1008
```

Then re-run tag.sh.

"Configured remote '...' does not exist"

You passed --remote <name> but no such git remote is set up. Check with git remote -v and either add the remote or pick a different name.

Bump commit didn't happen

If `--no-bump` was passed, this is expected. Otherwise check that the writer functions actually changed the build files — the script verifies the write by re-reading the value, so a silent failure is unlikely; an outright `Failed to write ...` error is the usual signal.

One remote rejected the push

The script uses `set -e`, so it stops at the first failed push. The earlier remotes have already received the tag/commit; the failing one must be retried manually:

```
git push <bad-remote> --follow-tags
```

...or rerun with `--remote <bad-remote>` once the underlying issue (auth, network, branch protection) is fixed.

FAQ

Q: Why is the API versionCode separate from the Android versionCode? A: They are independent products with independent release cycles. Coupling them would force one to bump every time the other ships.

Q: Does the script ever force-push? A: No. It does `git push <remote> refs/tags/<tag>` and `git push <remote> HEAD:<branch>` — no + prefix, no `--force`. Tag movements and history rewrites are deliberately operator-only.

Q: Does the script touch other branches? A: No. It tags HEAD and pushes to the **current branch's** name on each remote. If you're not on master, you'll push to that branch on every remote, which is rarely what you want — usually you should release from master.

Q: Can I run it from CI? A: Yes; `--dry-run` makes for a safe preview job, and the real run can be gated behind a manual approval. Configure the CI runner with SSH keys for every upstream you want to push to.

Q: What about pre-release versions like 1.0.0-rc1? A: Not supported by the bump logic — the semver regex requires three numeric parts. Set such versions by hand and run with `--no-bump`.